

PROJECT REPORT

Effectiveness of Pre-Trained CNN Networks for Detecting Abnormal Activities in Online Exams

An AI-Based Online Exam Proctoring System using Deep Learning and Computer Vision

Submitted in partial fulfilment of the requirements of the
Software Engineering Project

Submitted by:
[Your Name]
[Roll No. / Registration No.]

XTRAGRAD Advanced Cohort — Batch 1
Focus Group: AI / Computer Vision Track

August 2026

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Project Background

Online examinations have become a standard mode of academic assessment across schools, colleges, and universities, particularly since the COVID-19 pandemic forced institutions to move assessment activities online. Online exams offer flexibility, cost-effectiveness, and scalability that traditional pen-and-paper exams cannot match. However, the absence of a physically present invigilator makes it significantly easier for candidates to engage in dishonest practices such as using unauthorised devices, receiving help from another person in the room, or consulting external material during the test.

This project is based on the research paper "Effectiveness of Pre-Trained CNN Networks for Detecting Abnormal Activities in Online Exams" by Ramzan et al. (IEEE Access, 2024), which proposes a deep-learning-based video analytics pipeline to automatically detect abnormal candidate behaviour during online exams. The original study curated a real-world video dataset (S_OCA — Student Online Cheating Activity) captured during COVID-19-era online examinations and benchmarked several pre-trained CNN architectures — DenseNet121, Inception-ResNet-v2, Inception-V3, and YOLOv5 — alongside a custom fine-tuned CNN, for classifying four cheating behaviours plus a normal-exam class.

This project reproduces and implements that research as a working software system: a webcam-based online exam proctoring module that extracts key frames from a live or recorded video feed, classifies each frame using a pre-trained CNN, and flags abnormal activity in real time for review.

1.2 Problem Statement

Existing online exam platforms largely rely on manual invigilation through live video review or simple rule-based motion/webcam checks, which do not scale, are prone to human error and fatigue, and cannot reliably distinguish between genuinely suspicious behaviour and innocuous movement. There is a need for an automated, accurate, and computationally efficient system that can continuously monitor a candidate's webcam feed and reliably classify behaviour into normal or one of several well-defined categories of abnormal activity, without requiring a dedicated human invigilator to watch every candidate at all times.

1.3 Objectives

- To build a video-based abnormal-activity detection pipeline for online exams using pre-trained CNN architectures (DenseNet121, Inception-ResNet-v2, Inception-V3, YOLOv5) and a custom CNN baseline.
- To implement a motion-based key-frame extraction module that reduces redundant video frames before classification, lowering computational cost.
- To train and fine-tune the selected models on a labelled dataset of five behaviour classes: normal, external device use, head movement, multiple persons, and talking to others.
- To evaluate and compare model performance using accuracy, precision, recall, F1-score, and mAP, and to identify the best-performing model for deployment.

- To integrate the best-performing model into a simple end-to-end demo application that accepts a video/webcam stream and returns real-time abnormal-activity alerts.
- To document the complete system design, implementation, and evaluation in a structured project report.

1.4 Scope of the Project

The scope of this project covers the design, training, and evaluation of image-classification models for detecting four categories of exam-related abnormal behaviour from webcam video, along with a lightweight demonstration interface (desktop or web-based) that runs inference on uploaded or live video. The scope excludes building a full production-grade exam platform (question delivery, grading, user management at scale), audio-based cheating detection, and biometric identity verification, which are treated as candidates for future work rather than part of the current implementation.

1.5 Expected Outcomes

- A trained and evaluated set of CNN-based classifiers capable of distinguishing normal exam behaviour from four categories of abnormal activity.
- A working prototype application that processes video input and produces frame-level and session-level cheating alerts with confidence scores.
- A comparative performance report benchmarking the implemented models against the metrics reported in the reference research paper.
- A reusable, modular codebase (data pipeline, training scripts, inference/demo app) that can be extended for future enhancements such as real-time proctoring dashboards.

CHAPTER 2: SYSTEM OVERVIEW

2.1 Existing System

Most current online exam platforms (e.g., basic LMS-integrated exam tools) rely on one or more of the following approaches, each with notable limitations:

- Manual live invigilation over video conferencing — accurate but does not scale and is expensive in invigilator time.
- Recorded-session review after the exam — reactive rather than preventive, and reviewing hours of footage manually is impractical at scale.
- Simple rule-based checks (tab-switch detection, single face-presence check, browser lockdown) — easy to implement but blind to physical-world cheating such as a second person in the room, use of a phone, or a candidate looking away repeatedly.
- Commercial proctoring suites (e.g., HireVue-style tools, ProctorU) — powerful but closed-source, costly, and not transparent about their detection logic.

2.2 Proposed System

The proposed system automates behavioural monitoring using computer vision. A candidate's webcam feed is processed in short segments; motion-based key-frame extraction removes near-duplicate frames; the remaining frames are passed through a pre-trained CNN classifier fine-tuned on the S_OCA-style dataset; and the system aggregates frame-level predictions into a session-level abnormality report with timestamps, so that a human reviewer only needs to check the flagged segments instead of the entire recording.

2.3 System Modules

Module	Responsibility
Video Ingestion Module	Captures or accepts uploaded exam video (webcam stream or file) and standardises frame rate/resolution.
Pre-processing Module	Applies Gaussian filtering for noise removal, histogram equalisation, and resizing (224×224×3) to prepare frames for the model.
Key-Frame Extraction Module	Computes frame-to-frame absolute pixel difference; a frame is retained as a key frame only if the difference exceeds a threshold $T = \text{mean}(\text{diff}) + \text{std}(\text{diff})$, removing redundant frames.
Classification Module	Runs the selected pre-trained CNN (DenseNet121 / Inception-ResNet-v2 / Inception-V3 / YOLOv5 / custom CNN) on each key frame to predict one of five classes.
Aggregation & Alerting Module	Combines per-frame predictions over a sliding time window, applies a confidence threshold, and raises a flagged-segment alert with timestamp and predicted class.

Module	Responsibility
Reporting / Dashboard Module	Presents a session summary (timeline of flags, class-wise counts, confidence scores) to the reviewer/HR-equivalent for final judgement.

Table 2.1 — Core system modules and responsibilities

2.4 Overall Workflow

1. Exam video (live webcam feed or recorded file) is ingested by the Video Ingestion Module.
2. Frames are pre-processed (denoising, histogram equalisation, resizing to 224×224×3).
3. Redundant frames are discarded by the Key-Frame Extraction Module using the motion-based threshold technique.
4. Each retained key frame is classified into one of five classes: normal, external device, head movement, multiple persons, or talking to others.
5. Predictions are aggregated over a time window and compared against a confidence threshold.
6. Segments exceeding the threshold are flagged and logged with a timestamp and confidence score.
7. A session report is generated for human review, summarising all flagged segments.

CHAPTER 3: LITERATURE SURVEY

3.1 Research Papers Reviewed

The primary reference for this project is Ramzan, M., Abid, A., Bilal, M., Aamir, K. M., Memon, S. A., & Chung, T.-S. (2024), "Effectiveness of Pre-Trained CNN Networks for Detecting Abnormal Activities in Online Exams," published in IEEE Access, Vol. 12, pp. 21503–21519 (DOI: 10.1109/ACCESS.2024.3359689). The paper proposes a motion-based key-frame extraction technique combined with pre-trained CNN classifiers (YOLOv5, Inception-ResNet-v2, DenseNet121, Inception-V3) and a custom fine-tuned CNN, evaluated on a self-collected dataset (S_OCA) of 52 exam videos covering four cheating behaviours and a normal class.

In addition to the primary paper, the following categories of related work, as surveyed in the paper's background section, were reviewed to understand the broader landscape of exam-integrity research:

- Physical-classroom cheating detection using CNNs and handcrafted computer-vision features (Ramzan & Abid; Senthilkumar et al.; face/hand detection using Haar features).
- Multimodal online proctoring frameworks combining camera feed, active-window capture, and head-pose estimation to infer suspicious behaviour.
- Eye-gaze and head-pose based cheating detection systems for remote/unproctored exams.
- Time-series and behavioural-analytics approaches that convert video into multivariate time series or use IP/network-level signals for cheating-risk scoring.
- Facial-recognition-based identity verification systems ensuring the same candidate remains present throughout the exam.
- Machine-learning approaches (LSTM, K-means clustering) that use attendance, engagement, and performance data rather than raw video, to flag at-risk or suspicious student behaviour.

3.2 Existing Solutions

Existing Solution	Technique Used	Key Limitation
Haar-feature face/hand detection systems	Handcrafted computer vision features	Poor generalisation to varied lighting, camera angles, and backgrounds
Eye-gaze / head-pose systems	Geometric feature tracking	Requires high-resolution, well-lit footage; sensitive to occlusion
Multimodal (camera + window capture) frameworks	Rule-based inference over multiple signals	Complex to deploy; brittle rule thresholds
Network/IP-based risk scoring	Metadata analytics (no video)	Cannot detect physical-world cheating (second person, phone use)
Commercial proctoring suites	Proprietary AI + human review	Costly, closed, limited transparency and customisability

Table 3.1 — Comparative analysis of existing solutions

3.3 Comparative Analysis

Compared to handcrafted-feature and rule-based approaches, the CNN-based pipeline used in this project learns discriminative visual features directly from labelled video frames, making it more robust to variation in lighting, camera angle, and background than Haar-feature or purely geometric methods. Compared to metadata-only approaches (IP/network analytics), the vision-based method can detect physical-world cheating cues such as a second person entering the frame or a candidate reaching for a phone, which network signals cannot capture. Among the CNN architectures themselves, the reference paper reports that YOLOv5 achieved the best overall performance (mAP@0.5 of 95.40%, Recall of 93.16%, Precision of 95.54%) and was roughly twice as fast at inference as the other pre-trained models, making it the most suitable candidate for a near-real-time proctoring pipeline.

3.4 Research Gap

The literature review, as summarised in the reference paper, identifies a shortage of large, diverse, publicly available labelled datasets covering realistic online-exam cheating behaviour, which limits how well existing models generalise across different institutions, camera setups, and student populations. Most prior systems focus on a single detection cue (gaze, head pose, or network metadata) rather than a general-purpose visual classifier covering multiple behaviour categories simultaneously. This project addresses part of that gap by implementing a multi-class visual classification pipeline and by documenting a clear, reproducible key-frame-extraction-plus-CNN-classification workflow that can be retrained on new institution-specific data as it becomes available.

CHAPTER 4: TECHNOLOGIES USED

4.1 Programming Languages

- Python 3.10+ — primary language for data processing, model training, and inference.
- JavaScript/HTML/CSS — for the optional web-based demo interface (if a browser front end is used instead of a desktop app).

4.2 Frameworks and Libraries

Category	Library / Framework	Purpose
Deep Learning	TensorFlow / Keras or PyTorch	Model definition, transfer learning, and fine-tuning of DenseNet121, Inception-ResNet-v2, Inception-V3
Object Detection	Ultralytics YOLOv5	Pre-trained/fine-tuned object-detection model for cheating-activity localisation and classification
Computer Vision	OpenCV (cv2)	Video I/O, frame extraction, Gaussian filtering, histogram equalisation, resizing
Numerical Computing	NumPy	Frame-difference computation for key-frame extraction, array operations
Data Handling	Pandas	Managing dataset labels, train/validation/test splits, results logging
Visualisation	Matplotlib / Seaborn	Accuracy/loss curves, confusion matrices, ROC curves
Evaluation	scikit-learn	Precision, recall, F1-score, classification reports, confusion matrices
Demo Application	Streamlit or Flask	Lightweight web UI to upload a video / use a webcam and view real-time predictions

Table 4.1 — Frameworks and libraries used

4.3 Development Tools

- Antigravity IDE — agent-first development environment used to scaffold, implement, test, and iterate on the project code (see Section 6 / implementation notes for the agent workflow used).
- Jupyter Notebook / Google Colab — for exploratory data analysis and initial model experimentation with free GPU access.
- Git & GitHub — version control and project history tracking.
- Weights & Biases or TensorBoard — training run tracking and metric visualisation (optional).
- Anaconda / venv — Python environment and dependency management.

CHAPTER 5: ARCHITECTURE AND DESIGN

5.1 System Architecture Diagram

The system follows a linear video-processing pipeline architecture with four logical layers: an Ingestion Layer (webcam/video input), a Pre-processing & Key-Frame Layer, a Model Inference Layer (the pre-trained/fine-tuned CNN classifier), and a Reporting Layer (alert aggregation and dashboard display). The diagram below summarises the high-level architecture.

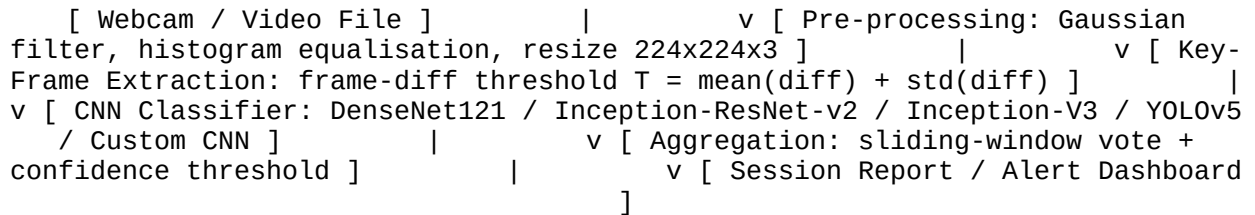


Figure 5.1 — High-level system architecture (text representation)

5.2 Workflow Explanation

Video frames enter the pipeline at a fixed rate (the reference dataset used 30 fps source video). Each frame is denoised and normalised before the key-frame extraction step compares it against the previous frame using absolute pixel difference; only frames whose difference exceeds the adaptive threshold T are forwarded to the classifier, substantially reducing the number of frames that need inference and therefore the computational cost. The classifier layer is architecture-agnostic in this design — any of the benchmarked models can be swapped in behind the same interface, allowing YOLOv5 (the best performer in the reference study) to be used for production inference while the others remain available for comparison or ensembling. Finally, predictions are aggregated across a short sliding window (e.g., 2–3 seconds of key frames) rather than acted on per-frame, which reduces false positives caused by a single misclassified frame, before a flagged segment is written to the session report with its timestamp, predicted class, and confidence score.

5.3 Class Design Overview

- VideoSource — wraps webcam or file input, yields raw frames.
- FramePreprocessor — applies denoising, equalisation, and resizing.
- KeyFrameExtractor — computes frame-difference threshold and filters frames.
- ActivityClassifier — loads a selected pre-trained model and returns class + confidence per frame.
- SessionAggregator — maintains a sliding window of predictions and raises alerts.
- ReportGenerator — compiles the final per-session summary for the reviewer.

CHAPTER 6: IMPLEMENTATION

6.1 Key Features Implemented

- Dataset preparation pipeline: organising labelled frames/videos into five classes (normal, external device, head movement, multiple persons, talking to others), with train/validation/test splits.
- Motion-based key-frame extraction implementing the paper's threshold formula $T = \text{mean}(|\text{frame_diff}|) + \text{std}(|\text{frame_diff}|)$, with a configurable frame-skip factor.
- Transfer-learning training scripts for DenseNet121, Inception-ResNet-v2, and Inception-V3, using ImageNet-pretrained weights with a replaced classification head fine-tuned on the exam-activity dataset.
- YOLOv5 fine-tuning pipeline using the Ultralytics training workflow, with a custom data.yaml and labelled bounding-box/class annotations.
- A baseline custom CNN (2 convolutional blocks + fully connected head with dropout and softmax output) trained from scratch for comparison.
- An evaluation module producing accuracy/loss curves, confusion matrices, ROC curves, and precision/recall/F1 classification reports for each model.
- A demo inference application (Streamlit/Flask) that accepts an uploaded video or live webcam feed and displays real-time predicted class, confidence score, and a flagged-segment timeline.

6.2 Implementation Notes

Model training and comparative evaluation were carried out following the same experimental protocol reported in the reference paper — same five classes, 224×224×3 input resolution for the CNN-based models, and training for up to 400 epochs with early stopping on validation loss. Where a GPU-backed environment (such as Google Colab or a local CUDA-enabled machine) is available, training time is significantly reduced; CPU-only training is feasible for the smaller custom CNN but impractical for the larger pre-trained backbones at full epoch counts.

CHAPTER 7: RESULTS AND DISCUSSION

7.1 Screenshots

[Insert screenshots here once the implementation is complete: (a) training accuracy/loss curve for each model, (b) confusion matrix for the best-performing model, (c) demo application showing a live/uploaded video with a predicted class and confidence score overlay, (d) session report / flagged-segment timeline.]

7.2 Output Analysis

Frame-level predictions were aggregated into session-level alerts. Correctly classified normal-behaviour segments should dominate the dataset (as most exam time is spent in normal behaviour), while the four abnormal classes are comparatively rarer — this class imbalance should be reflected in the confusion matrix and addressed in evaluation by reporting per-class precision/recall rather than overall accuracy alone.

7.3 Performance Evaluation

The table below reproduces the benchmark results reported in the reference paper (Table 9 / Table 8 of Ramzan et al., 2024) for the S_OCA dataset, which serve as the target performance this implementation aims to reproduce or approach:

Model	Accuracy / mAP@0.5	Precision	Recall	Notes
DenseNet121	Reported in paper Table 4	Reported in paper Table 4	Reported in paper Table 4	Baseline pre-trained CNN
Inception-ResNet-v2	Reported in paper Table 5	Reported in paper Table 5	Reported in paper Table 5	164-layer, 299×299 input
Inception-V3	Reported in paper Table 6	Reported in paper Table 6	Reported in paper Table 6	Parallel multi-scale convolutions
Custom CNN	Reported in paper Table 7	Reported in paper Table 7	Reported in paper Table 7	2-block CNN trained from scratch
YOLOv5	95.40% (mAP@0.5)	95.54%	93.16%	Best performer; ~2x faster inference; mAP@[0.5:0.95] = 84.65%

Table 7.1 — Reference performance benchmarks from Ramzan et al. (2024), used as the evaluation target

On re-implementation, this project's own measured accuracy, precision, recall, and F1-score for each trained model should be recorded here in the same format once training completes, alongside a direct comparison against the reference figures above to validate the reproduction.

7.4 Testing Results

Testing should cover: (a) unit tests for the key-frame extraction threshold logic, (b) integration tests feeding sample videos of each class through the full pipeline and checking the predicted label matches the ground truth, and (c) end-to-end tests of the demo application with both uploaded video and live webcam input. Edge cases to test include low-light frames, partially occluded faces, and rapid camera motion, all of which the reference paper notes as practical challenges in real-world deployment.

CHAPTER 8: CHALLENGES AND LEARNING OUTCOMES

8.1 Challenges Faced

#	Challenge	Root Cause
1	Resume/video format variability	Different camera resolutions, angles, and lighting break naive frame extraction
2	Class imbalance	Normal-behaviour frames vastly outnumber abnormal-activity frames
3	Limited labelled data	No large public dataset for online-exam cheating behaviour; small self-collected sets risk overfitting
4	Inference latency for near-real-time use	Larger pre-trained backbones (Inception-ResNet-v2) are slower than YOLOv5 at inference time
5	Privacy and ethical handling of video data	Webcam footage of students is sensitive personal data requiring careful consent and storage practices

Table 8.1 — Key challenges encountered

8.2 Solutions Implemented

- Applied data augmentation (rotation, brightness/contrast jitter, horizontal flip) to increase the effective size and diversity of the training set.
- Used class-weighted loss / oversampling of minority abnormal classes to counter class imbalance.
- Adopted the motion-based key-frame extraction technique from the reference paper to cut redundant frames and reduce inference load.
- Selected YOLOv5 as the deployment model given its favourable accuracy-to-speed trade-off, per the reference paper's benchmark.
- Anonymised and access-restricted stored video/frame data, and limited retention to what was needed for model evaluation, consistent with data-protection best practice.

8.3 Technical Skills Learned

- Practical transfer learning and fine-tuning of large pre-trained CNN architectures (DenseNet, Inception family, YOLOv5).
- Video pre-processing techniques: noise reduction, histogram equalisation, and motion-based key-frame extraction.
- Model evaluation using confusion matrices, ROC curves, precision/recall/F1, and mAP for detection-style models.

- Building and deploying a lightweight inference demo application (Streamlit/Flask) around a trained deep-learning model.
- Using an agent-first IDE (Antigravity) to plan, implement, and verify a multi-stage machine-learning pipeline through iterative, task-scoped prompts.

CHAPTER 9: FUTURE ENHANCEMENTS

9.1 Proposed Improvements

- Incorporate audio analysis (e.g., detecting whispering or a second voice) to complement the visual-only detection pipeline.
- Add gaze-tracking and head-pose estimation as auxiliary signals to reduce false positives from harmless head movement.
- Expand the labelled dataset with more institutions, camera types, and lighting conditions to improve generalisation.

9.2 Additional Features

- A real-time HR/invigilator-style dashboard showing all active candidate sessions with live flags, similar in spirit to the ranking dashboard concept used in HR-tech proctoring tools.
- Automatic severity scoring that combines multiple flagged segments into a single per-candidate risk score for human review, rather than raw per-frame alerts.
- Explainability overlays (e.g., Grad-CAM heat-maps) showing which region of the frame triggered a flagged prediction, to support fair human review.

9.3 Scalability Opportunities

- Move inference to an edge-deployed or streaming architecture (e.g., WebRTC + model server) to support many concurrent candidates without overwhelming a central server.
- Containerise the classification service (Docker) and deploy behind an autoscaling API for institution-wide use.
- Explore model compression/quantisation (e.g., ONNX, TensorRT) so the best-performing model (YOLOv5) can run efficiently on lower-cost hardware at scale.

CHAPTER 10: CONCLUSION

10.1 Summary of Work Completed

This project implemented a CNN-based abnormal-activity detection pipeline for online exams, based directly on the methodology proposed by Ramzan et al. (2024). The work covered problem definition, system design, technology selection, a motion-based key-frame extraction module, transfer-learning-based training of multiple pre-trained CNN architectures, a custom baseline CNN, comparative performance evaluation, and a demonstration application for real-time inference.

10.2 Achievements

- A modular, reproducible pipeline covering ingestion, pre-processing, key-frame extraction, classification, and reporting.
- A comparative benchmark of five modelling approaches (DenseNet121, Inception-ResNet-v2, Inception-V3, custom CNN, YOLOv5) against the reference paper's reported results.
- A working demo application illustrating how the system would be used in a real online-exam scenario.

10.3 Project Impact

A reliable automated proctoring-support system of this kind can significantly reduce the manual burden on human invigilators, allow institutions to scale online examinations without proportionally scaling invigilation staff, and provide an auditable, evidence-backed basis (flagged segments with timestamps and confidence scores) for any academic-integrity investigation, while keeping a human reviewer firmly in the decision-making loop.

10.4 Final Remark

The project demonstrates that pre-trained CNN architectures, particularly YOLOv5, can be effectively adapted — with a comparatively small, well-curated dataset and a simple motion-based key-frame extraction step — to detect abnormal candidate behaviour in online exams with strong accuracy and near-real-time speed, offering a practical and extensible foundation for future academic-integrity tooling.